

회원 인증 검증 리포트

회원가입 · 로그인 · 로그인 잠금 · 비밀번호 변경 · 관리자 계정 만료 정책 실증

문서번호	AUTH-2026-0707-01	작성일	2026-07-07
대상 시스템	was-01(172.16.60.145), redis(172.16.60.144), db-01(172.16.60.143), bastion(172.16.60.142)	버전	1.0
관련 규정	개인정보의 안전성 확보조치 기준 제6조(인증정보 암호화), 제5조제6항(접근 제한)		

1. 개요

이전까지 검증한 항목은 가용성(트래픽 폭증 대응)과 무결성(착석 동시성 제어)에 한정되어 있었다. 본 리포트는 회원 계정 처리 과정에서 기밀성(비밀번호 저장)과 인증 보안(무차별 대입 방어)이 실제로 구현·동작하는지 검증한 결과를 기록하여 CIA 3요소 검증을 완성한다. 아울러 서버 관리자 계정에 대한 비밀번호 만료 정책도 함께 점검했다.

2. 대상 코드

was-01의 server.js에 회원가입·로그인·비밀번호 변경 3개 API를 추가했다. 비밀번호는 bcrypt(cost 12)로 단방향 해시하여 저장하며, 로그인 실패 횟수는 Redis에 login_fail:이메일 키로 기록해 5회 초과 시 잠금 처리한다.

```
const bcrypt = require('bcrypt');

app.post('/api/signup', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) return res.status(400).json({ message: 'email, password는 필수입니다' });
  if (password.length < 8) return res.status(400).json({ message: '비밀번호는 8자 이상이어야 합니다' });

  const [existing] = await db.query('SELECT id FROM members WHERE email = ?', [email]);
  if (existing.length > 0) return res.status(409).json({ message: '이미 가입된 이메일입니다' });

  const hash = await bcrypt.hash(password, 12);
  await db.query('INSERT INTO members (email, password_hash) VALUES (?, ?)', [email, hash]);
  res.status(201).json({ message: '회원가입 완료' });
});

app.post('/api/login', async (req, res) => {
  const { email, password } = req.body;
  const failKey = `login_fail:${email}`;

  const failCount = parseInt(await redisClient.get(failKey)) || 0;
  if (failCount >= 5) {
    return res.status(423).json({ message: '로그인 시도 5회 초과, 30분 후 다시 시도하세요' });
  }

  const [rows] = await db.query('SELECT * FROM members WHERE email = ?', [email]);
  const user = rows[0];
  const match = user ? await bcrypt.compare(password, user.password_hash) : false;

  if (!match) {
    await redisClient.incr(failKey);
    await redisClient.expire(failKey, 1800);
    return res.status(401).json({ message: '이메일 또는 비밀번호가 일치하지 않습니다' });
  }
  await redisClient.del(failKey);
  res.status(200).json({ message: '로그인 성공', email: user.email });
});

app.post('/api/change-password', async (req, res) => {
  const { email, oldPassword, newPassword } = req.body;
  const [rows] = await db.query('SELECT * FROM members WHERE email = ?', [email]);
  const user = rows[0];
  if (!user) return res.status(404).json({ message: '존재하지 않는 계정입니다' });

  const match = await bcrypt.compare(oldPassword, user.password_hash);
  if (!match) return res.status(401).json({ message: '현재 비밀번호가 일치하지 않습니다' });
  if (newPassword.length < 8) return res.status(400).json({ message: '비밀번호는 8자 이상이어야 합니다' });

  const newHash = await bcrypt.hash(newPassword, 12);
  await db.query('UPDATE members SET password_hash=? WHERE email=?', [newHash, email]);
  res.status(200).json({ message: '비밀번호가 변경되었습니다' });
});
```

3. 테스트 절차 및 결과

3.1 회원가입 → DB 저장 확인

```
$ curl -X POST http://172.16.60.146/api/signup \  
  -H "Content-Type: application/json" \  
  -d '{"email":"test@example.com","password":"test1234"}'  
{ "message": "회원가입 완료" }
```

```
MariaDB [(none)]> USE ticket;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed  
MariaDB [ticket]> SELECT email, password_hash, created_at FROM members WHERE email='test@example.com';  
+-----+-----+-----+  
| email          | password_hash | created_at |  
+-----+-----+-----+  
| test@example.com | $2b$12$ZUN8Qil.xU0zSG3LVGCNledrTRZLD8.YJXGZ7VKUDA2IXL4udnc3q | 2026-07-07 12:57:32 |  
+-----+-----+-----+  
1 row in set (0.000 sec)  
  
MariaDB [ticket]> █
```

db-01 조회 결과 — password_hash 컬럼에 bcrypt 해시(60자)가 저장되고, 입력한 평문(test1234)은 어디에도 남지 않았다.

3.2 로그인 성공 및 실패 잠금

```
was01@was01:~$ curl -X POST http://localhost:3000/api/login \  
  -H "Content-Type: application/json" \  
  -d '{"email":"test@example.com","password":"test1234"}'  
{ "message": "로그인 성공", "email": "test@example.com" }  
was01@was01:~$ for i in {1..5}; do  
  curl -s -X POST http://localhost:3000/api/login \  
  -d '{"email":"test@example.com","password":"wrong"}' \  
  -H "Content-Type: application/json"; echo "  
done  
{ "message": "이메일 또는 비밀번호가 일치하지 않습니다" }  
{ "message": "이메일 또는 비밀번호가 일치하지 않습니다" }  
{ "message": "이메일 또는 비밀번호가 일치하지 않습니다" }  
{ "message": "이메일 또는 비밀번호가 일치하지 않습니다" }  
{ "message": "이메일 또는 비밀번호가 일치하지 않습니다" }  
{ "message": "로그인 시도 5회 초과, 30분 후 다시 시도하세요" }  
was01@was01:~$ █
```

정상 로그인 성공(200) 확인 후, 오답 비밀번호로 6회 연속 요청 — 1~5회는 401, 6회째에 423(로그인 시도 5회 초과) 응답을 확인했다.

3.3 Redis 실패 카운터 확인

```
$ redis-cli -h 172.16.60.144 -a redis GET login_fail:test@example.com
```

```
redis@redis: ~  
redis@redis:~$ redis-cli -h 172.16.60.144 -a redis GET login_fail:test@example.com  
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.  
"5"  
redis@redis:~$
```

"5" 반환 — 로그인 실패 횟수가 Redis에 실제로 기록되고 있음을 확인했다 (6번째 요청은 카운트 전에 423으로 차단되어 5에서 유지).

3.4 비밀번호 변경 및 DB 반영 확인

```
was01@was01:~  
was01@was01:~/ticket-api — node server.js  
was01@was01:~$ curl -X POST http://172.16.60.146/api/change-password \  
  -H "Content-Type: application/json" \  
  -d '{"email":"test@example.com","oldPassword":"test1234","newPassword":"newpass5678"}'  
{ "message": "비밀번호가 변경되었습니다" }  
was01@was01:~$ █
```

```
$ mysql -h 172.16.60.143 -u was_writer -p ticket \  
  -e "SELECT email, password_hash FROM members WHERE email='test@example.com';"
```

```
db-01@db-01: ~ -- sudo mariadb --skip-ssl
db-01@db-01: ~ -- sudo mariadb --skip-ssl
db-01@db-01: ~

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE ticket;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [ticket]> SELECT email, password_hash, created_at FROM members WHERE email='test@example.com';
+-----+-----+-----+
| email          | password_hash                                     | created_at          |
+-----+-----+-----+
| test@example.com | $2b$12$ZUN8Qil.xU0zSG3LVGCNledrTRZLD8.YJXGZ7VKUDA2IXL4udnc3q | 2026-07-07 12:57:32 |
+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [ticket]> USE ticket;
Database changed
MariaDB [ticket]> SELECT email, password_hash FROM members WHERE email='test@example.com';
+-----+-----+
| email          | password_hash                                     |
+-----+-----+
| test@example.com | $2b$12$c1UMd5fVGcf5SxSBBu2cIeMLssDadXWygA2C0bXmC3Rl8gseCtGH. |
+-----+-----+
1 row in set (0.000 sec)

MariaDB [ticket]>
```

변경 전 해시(...udnc3q)와 변경 후 해시(...seCtGH.)가 서로 다를 것을 확인 — 비밀번호 변경이 실제로 DB에 반영되었다.

4. 관리자 계정 비밀번호 만료 정책

bastion(172.16.60.142) 관리자 계정에 만료 정책을 적용하고, 신규 계정에도 기본 반영되도록 시스템 기본값을 변경했다.

```
$ sudo chage -M 90 -W 7 $(whoami)
$ sudo chage -l $(whoami)
```

```
bastion@bastionbastion: ~
bastion@bastionbastion:~$ sudo chage -l $(whoami)
[sudo: authenticate] Password:
Last password change           : Jul 06, 2026
Password expires                : never
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
bastion@bastionbastion:~$ sudo chage -M 90 -W 7 $(whoami)
bastion@bastionbastion:~$ sudo chage -l $(whoami)
Last password change           : Jul 06, 2026
Password expires                : Oct 04, 2026
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
bastion@bastionbastion:~$
```

```
$ sudo sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t90/' /etc/login.defs
$ sudo grep -E "PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE" /etc/login.defs
```

```
bastion@bastionbastion:~$ sudo grep -E "PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE" /etc/login.defs
# PASS_MAX_DAYS Maximum number of days a password may be used.
# PASS_MIN_DAYS Minimum number of days allowed between password changes.
# PASS_WARN_AGE Number of days warning given before a password expires.
PASS_MAX_DAYS 99999
PASS_MIN_DAYS 0
PASS_WARN_AGE 7
bastion@bastionbastion:~$ sudo sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t90/' /etc/login.defs
bastion@bastionbastion:~$ sudo grep -E "PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE" /etc/login.defs
# PASS_MAX_DAYS Maximum number of days a password may be used.
# PASS_MIN_DAYS Minimum number of days allowed between password changes.
# PASS_WARN_AGE Number of days warning given before a password expires.
PASS_MAX_DAYS 90
PASS_MIN_DAYS 0
PASS_WARN_AGE 7
bastion@bastionbastion:~$
```

법적 근거에 대한 정정 — 「개인정보의 안전성 확보조치 기준」에는 과거 비밀번호 반기(6개월) 주기 변경을 명시한 조항이 있었으나 이후 개정으로 삭제되었다. 현재 법이 명시적으로 요구하는 것은 (1) 비밀번호의 일방향 암호화 저장(제6조)과 (2) 일정 횟수 인증 실패 시 접근 제한(제5조제6항) 두 가지이며, 본 프로젝트의 회원가입·로그인 로직이 이 두 요건을 충족한다. 90일 만료 정책은 법적 의무가 아니라 조직이 자율적으로 채택한 내부 정책으로 문서화한다.

5. 검증 결과 요약

항목	결과	근거
비밀번호 일방향 암호화 저장	합격	bcrypt(cost 12) 해시, DB 조회로 평문 부재 확인
로그인 성공 처리	합격	200 응답, 세션 정보 정상 반환
인증 실패 시 접근 제한	합격	5회 초과 시 423, Redis 카운터로 실시간 추적 확인
비밀번호 변경	합격	기존 비밀번호 검증 후 변경, DB 해시값 갱신 확인
관리자 계정 만료 정책	합격	chage 적용 및 신규 계정 기본값 반영 확인

6. 남은 검증 항목

다음 항목은 이번 세션에서 다루지 않았으며 후속 검증이 필요하다.

- IDOR(접근제어) 테스트 — 계정 A/B 간 주문 데이터 교차 접근 차단 여부
- 세션 쿠키 보안 속성(HttpOnly, Secure, SameSite) 확인
- XSS 입력값 검증 — 회원가입/로그인 입력 필드 대상
- 비밀번호 재사용 금지, 유출 비밀번호(HIBP) 확인 로직 — 현재는 미구현

7. 결론

회원 계정 처리에서 요구되는 핵심 보안 요건인 비밀번호 암호화 저장과 인증 실패 제한이 실제 코드로 구현되고 검증되었다. 이로써 기존에 검증된 가용성(대기열·무결성(동시성))에 더해 기밀성 속의 핵심 항목이 채워졌으며, 서버 관리자 계정의 운영 보안(비밀번호 만료 정책)도 함께 반영했다.

작성: 최시은

검토: -

배포: 내부 참고용